10 · Reports

Totals for the catchment, by day and by locality, with no names.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The **Reports** page shows, for 7, 30, 90 or 180 days: readings, citizens who submitted, flags and the flag rate; a bar chart of readings per day; and a table per locality. It never shows names.

Small localities are merged: any locality with fewer than 3 citizens is folded into "Other localities", because "1 of 1 citizens in Luz is flagged" would identify that person.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/phc.py          GET /phc/reports
└── services/localities.py     locality_of(), suppress() (the small-locality merge), MIN_CELL = 3
frontend/src/
├── pages/phc/Reports.tsx      the page (Recharts bar chart + table)
└── services/phc.ts            getReports(days)
```

How the pieces connect

1. The page reads `?days=` from the URL and calls `GET /phc/reports?days=30`.
2. The backend counts per local day (India time) and per locality, then merges small localities with `suppress()`.
3. The page draws the totals, the daily chart and the locality table.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/services/localities.py` · suppress

```
35  def suppress(rows: dict[str, dict[str, float]], size_key: str = "citizens") -> dict[str, dict[str, float]]:
36      """Fold localities below MIN_CELL into OTHER, summing their counts. Drops OTHER if it is itself too smal
            l."""
37      kept: dict[str, dict[str, float]] = {}
38      other: dict[str, float] = {}
39      for name, values in rows.items():
40          if values[size_key] >= MIN_CELL:
41              kept[name] = values
42          else:
43              for key, value in values.items():
44                  other[key] = other.get(key, 0) + value
45      if other and other[size_key] >= MIN_CELL:
46          kept[OTHER] = other
47      return kept
```

What the code means

- `rows` maps locality name to its counts. Localities with at least `MIN_CELL` citizens are kept as they are.
- Smaller ones are added together into `other` (`other.get(key, 0) + value` starts at 0 for a new key).
- If even "Other localities" is too small, it is dropped completely.

2. `backend/app/api/routes/phc.py` · reports (daily counts)

```
755 async def reports(
756     days: int = Query(30, ge=7, le=180),
757     worker: User = Depends(require_catchment_worker),
758     db: AsyncSession = Depends(get_session),
759 ) -> ReportsOut:
760     """Whole local days: 'last 30 days' is today and the 29 before it."""
761     now = now_utc()
762     first_day = local_date(now) - timedelta(days=days - 1)
763     start = local_day_start(first_day)
764     members = catchment_ids(worker)
765     flagged = func.count().filter(ProcessedData.is_anomaly.is_(True))
766     window_readings = (
767         select(HealthRecord.user_id)
768         .outerjoin(ProcessedData, ProcessedData.health_record_id == HealthRecord.id)
769         .where(HealthRecord.user_id.in_(members), HealthRecord.timestamp >= start, HealthRecord.timestamp <=
                now)
770     )
771
772     day = _local_day(HealthRecord.timestamp)
773     per_day = {
774         d: (n, f)
775         for d, n, f in (
776             await db.execute(window_readings.with_only_columns(day, func.count(), flagged).group_by(day))
777         ).all()
778     }
779     daily = [
780         DailyVolume(date=d, readings=per_day.get(d, (0, 0))[0], flags=per_day.get(d, (0, 0))[1])
781         for d in (first_day + timedelta(days=i) for i in range(days))
782     ]
```

What the code means

- "Last 30 days" means whole local days: today and the 29 before it (`local_date(now) - timedelta(days=days - 1)`).
- `func.count().filter(ProcessedData.is_anomaly.is_(True))` counts only flagged rows (SQL `COUNT(*) FILTER (WHERE ...)`).
- `window_readings.with_only_columns(day, func.count(), flagged).group_by(day)` reuses the same base query with different columns.
- The list comprehension fills every day in the window, using `(0, 0)` for days with no readings, so the chart has no gaps.

3. `frontend/src/pages/phc/Reports.tsx` · Reports (top of the page)

```
13 export function Reports() {
14   const { params, set } = useQueryState();
15   const days = Number(oneOf(params.get("days"), WINDOWS, "30"));
16   const rep = useResource((s) => getReports(days, s), String(days));
17   const blocked = workerState(rep);
18   if (blocked) return blocked;
19   const r = rep.data;
20
21   return (
22     <>
23       <PageHeader
24         title="Reports"
25         lead={r ? `${fmtDate(r.window.start)} – ${fmtDate(r.window.end)} (whole days, India time).` : undefi
               ned}
26         actions={
27           <select aria-label="Period" value={String(days)} onChange={(e) => set({ days: e.target.value })} c
                 lassName="min-h-11 rounded-xl border border-line bg-surface px-3 text-sm">
28             {WINDOWS.map((w) => <option key={w} value={w}>Last {w} days</option>)}
29           </select>
30         }
31       />
32       {!r ? <Loading /> : (
```

What the code means

- `Number(oneOf(params.get("days"), WINDOWS, "30"))` reads the period from the URL, allowing only 7, 30, 90 or 180.

- `actions={<select .../>}` passes a whole element as a prop to `PageHeader`, which places it on the right.
- `{WINDOWS.map((w) => <option key={w} value={w}>Last {w} days</option>)}` builds the dropdown from the list.

Syntax used in this feature

Code	What it means
`func.count().filter(cond)`	Count only rows that match cond.
`q.with_only_columns(a, b)`	Same query, different selected columns.
`dict.get(key, 0)`	The value, or 0 when the key is missing.